

Sales & Marketing Campaign Response Prediction

1. Problem Statement

A retail company frequently runs marketing campaigns to promote product categories. However, customers respond to these campaigns, leading to wasted budget and low ROI.

The goal is to predict whether a customer will accept the next marketing campaign using demog details, purchase history, and previous responses.

2. Business Goal

- Build a classification model to estimate the probability of campaign acceptance.
- Use insights to improve targeting strategy, reduce costs, and increase ROI.

3. Data Dictionary

Feature	Description
AcceptedCmp1–5	1 if customer accepted respective campaign, 0 otherwise
Response (Target)	1 if customer accepted the latest campaign, 0 otherwise
Complain	1 if customer complained in last 2 years
DtCustomer	Date of enrollment with company
Education	Education level of customer
Marital	Marital status
Kidhome	Number of small children
Teenhome	Number of teenagers
Income	Yearly household income
MntFishProducts	Amount spent on fish products (last 2 years)
MntMeatProducts	Amount spent on meat products (last 2 years)
MntFruits	Amount spent on fruit products (last 2 years)
MntSweetProducts	Amount spent on sweet products (last 2 years)
MntWines	Amount spent on wines (last 2 years)
MntGoldProds	Amount spent on gold products (last 2 years)
NumDealsPurchases	Purchases made with discount
NumCatalogPurchases	Purchases via catalog
NumStorePurchases	Purchases in store
NumWebPurchases	Purchases on website
NumWebVisitsMonth	Website visits in last month
Recency	Days since last purchase

Exploratory Data Analysis (EDA)

In [7]:

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier, GradientBoos
from sklearn.metrics import accuracy_score, classification_report, confusion_mat
import warnings
warnings.filterwarnings("ignore")
from imblearn.over_sampling import SMOTE

```

In [5]: `df = pd.read_csv(r"C:\Users\karan\Downloads\marketing_campaign.csv", sep = ';' ,`

In [7]: `df`

Out[7]:

	ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer
0	5524	1957	Graduation	Single	58138.0	0	0	2012-09-0
1	2174	1954	Graduation	Single	46344.0	1	1	2014-03-0
2	4141	1965	Graduation	Together	71613.0	0	0	2013-08-2
3	6182	1984	Graduation	Together	26646.0	1	0	2014-02-1
4	5324	1981	PhD	Married	58293.0	1	0	2014-01-1
...	...	...	...	...	...	...	...	...
2235	10870	1967	Graduation	Married	61223.0	0	1	2013-06-1
2236	4001	1946	PhD	Together	64014.0	2	1	2014-06-1
2237	7270	1981	Graduation	Divorced	56981.0	0	0	2014-01-2
2238	8235	1956	Master	Together	69245.0	0	1	2014-01-2
2239	9405	1954	PhD	Married	52869.0	1	1	2012-10-1

2240 rows × 29 columns

In [9]: `df.shape #to check how many rows and columns`

Out[9]: (2240, 29)

In [11]: `df.size #to check to total size of df`

Out[11]: 64960

In [13]: `df.columns #these are the total columns`

```
Out[13]: Index(['ID', 'Year_Birth', 'Education', 'Marital_Status', 'Income', 'Kidhome',
               'Teenhome', 'Dt_Customer', 'Recency', 'MntWines', 'MntFruits',
               'MntMeatProducts', 'MntFishProducts', 'MntSweetProducts',
               'MntGoldProds', 'NumDealsPurchases', 'NumWebPurchases',
               'NumCatalogPurchases', 'NumStorePurchases', 'NumWebVisitsMonth',
               'AcceptedCmp3', 'AcceptedCmp4', 'AcceptedCmp5', 'AcceptedCmp1',
               'AcceptedCmp2', 'Complain', 'Z_CostContact', 'Z_Revenue', 'Response'],
              dtype='object')
```

```
In [15]: df.info() #info is useful to check the dtype of every column
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 29 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                    2240 non-null   int64
1   Year_Birth                           2240 non-null   int64
2   Education                             2240 non-null   object
3   Marital_Status                       2240 non-null   object
4   Income                               2216 non-null   float64
5   Kidhome                              2240 non-null   int64
6   Teenhome                             2240 non-null   int64
7   Dt_Customer                          2240 non-null   object
8   Recency                              2240 non-null   int64
9   MntWines                             2240 non-null   int64
10  MntFruits                            2240 non-null   int64
11  MntMeatProducts                      2240 non-null   int64
12  MntFishProducts                      2240 non-null   int64
13  MntSweetProducts                     2240 non-null   int64
14  MntGoldProds                        2240 non-null   int64
15  NumDealsPurchases                    2240 non-null   int64
16  NumWebPurchases                      2240 non-null   int64
17  NumCatalogPurchases                  2240 non-null   int64
18  NumStorePurchases                    2240 non-null   int64
19  NumWebVisitsMonth                    2240 non-null   int64
20  AcceptedCmp3                         2240 non-null   int64
21  AcceptedCmp4                         2240 non-null   int64
22  AcceptedCmp5                         2240 non-null   int64
23  AcceptedCmp1                         2240 non-null   int64
24  AcceptedCmp2                         2240 non-null   int64
25  Complain                             2240 non-null   int64
26  Z_CostContact                        2240 non-null   int64
27  Z_Revenue                            2240 non-null   int64
28  Response                             2240 non-null   int64
dtypes: float64(1), int64(25), object(3)
memory usage: 507.6+ KB
```

```
In [17]: df.isnull().sum() #to check the null values in df now we have to treat it with n
```

```
Out[17]: ID 0
Year_Birth 0
Education 0
Marital_Status 0
Income 24
Kidhome 0
Teenhome 0
Dt_Customer 0
Recency 0
MntWines 0
MntFruits 0
MntMeatProducts 0
MntFishProducts 0
MntSweetProducts 0
MntGoldProds 0
NumDealsPurchases 0
NumWebPurchases 0
NumCatalogPurchases 0
NumStorePurchases 0
NumWebVisitsMonth 0
AcceptedCmp3 0
AcceptedCmp4 0
AcceptedCmp5 0
AcceptedCmp1 0
AcceptedCmp2 0
Complain 0
Z_CostContact 0
Z_Revenue 0
Response 0
dtype: int64
```

```
In [21]: df["Response"].value_counts()
```

```
Out[21]: Response
0      1906
1       334
Name: count, dtype: int64
```

```
In [14]: df.duplicated().sum() #to check duplicated values in df
```

```
Out[14]: 0
```

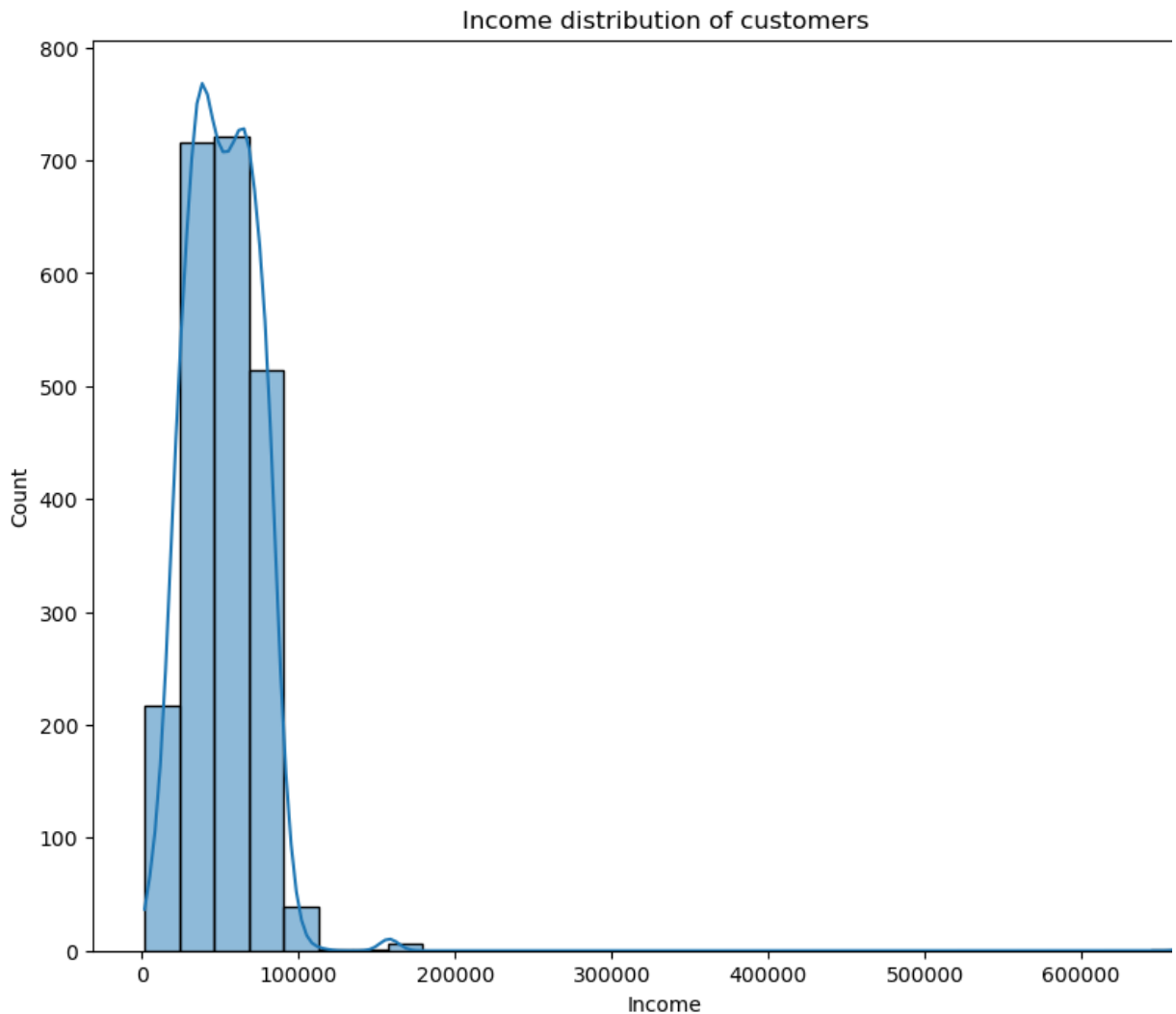
```
In [15]: df.describe().T #from here we can understand the numeric data and treat it if ne
```

Out[15]:		count	mean	std	min	25%	50%	75%
	ID	2240.0	5592.159821	3246.662198	0.0	2828.25	5458.5	8427.7
	Year_Birth	2240.0	1968.805804	11.984069	1893.0	1959.00	1970.0	1977.0
	Income	2216.0	52247.251354	25173.076661	1730.0	35303.00	51381.5	68522.0
	Kidhome	2240.0	0.444196	0.538398	0.0	0.00	0.0	1.0
	Teenhome	2240.0	0.506250	0.544538	0.0	0.00	0.0	1.0
	Recency	2240.0	49.109375	28.962453	0.0	24.00	49.0	74.0
	MntWines	2240.0	303.935714	336.597393	0.0	23.75	173.5	504.0
	MntFruits	2240.0	26.302232	39.773434	0.0	1.00	8.0	33.0
	MntMeatProducts	2240.0	166.950000	225.715373	0.0	16.00	67.0	232.0
	MntFishProducts	2240.0	37.525446	54.628979	0.0	3.00	12.0	50.0
	MntSweetProducts	2240.0	27.062946	41.280498	0.0	1.00	8.0	33.0
	MntGoldProds	2240.0	44.021875	52.167439	0.0	9.00	24.0	56.0
	NumDealsPurchases	2240.0	2.325000	1.932238	0.0	1.00	2.0	3.0
	NumWebPurchases	2240.0	4.084821	2.778714	0.0	2.00	4.0	6.0
	NumCatalogPurchases	2240.0	2.662054	2.923101	0.0	0.00	2.0	4.0
	NumStorePurchases	2240.0	5.790179	3.250958	0.0	3.00	5.0	8.0
	NumWebVisitsMonth	2240.0	5.316518	2.426645	0.0	3.00	6.0	7.0
	AcceptedCmp3	2240.0	0.072768	0.259813	0.0	0.00	0.0	0.0
	AcceptedCmp4	2240.0	0.074554	0.262728	0.0	0.00	0.0	0.0
	AcceptedCmp5	2240.0	0.072768	0.259813	0.0	0.00	0.0	0.0
	AcceptedCmp1	2240.0	0.064286	0.245316	0.0	0.00	0.0	0.0
	AcceptedCmp2	2240.0	0.013393	0.114976	0.0	0.00	0.0	0.0
	Complain	2240.0	0.009375	0.096391	0.0	0.00	0.0	0.0
	Z_CostContact	2240.0	3.000000	0.000000	3.0	3.00	3.0	3.0
	Z_Revenue	2240.0	11.000000	0.000000	11.0	11.00	11.0	11.0
	Response	2240.0	0.149107	0.356274	0.0	0.00	0.0	0.0

In [16]: `df['Income'].skew()` *#skewness is very high, which means it's right-skewed.*

Out[16]: 6.763487372811116

```
In [17]: plt.figure(figsize=(10,8))
sns.histplot(df["Income"], bins = 30, kde = True)
plt.title("Income distribution of customers")
plt.xlabel("Income")
plt.ylabel("Count")
plt.show()
```



```
In [18]: df['Income'] = df['Income'].fillna(df['Income'].median()) #this will fill the Na
```

```
In [19]: df['income'] = np.log1p(df['Income']) #we use log transformation method to remov
```

```
In [20]: df['income'].skew() #now the data becomes left skewed which it is not the best t
```

```
Out[20]: -1.1795023330987366
```

```
In [21]: from scipy.stats import boxcox
```

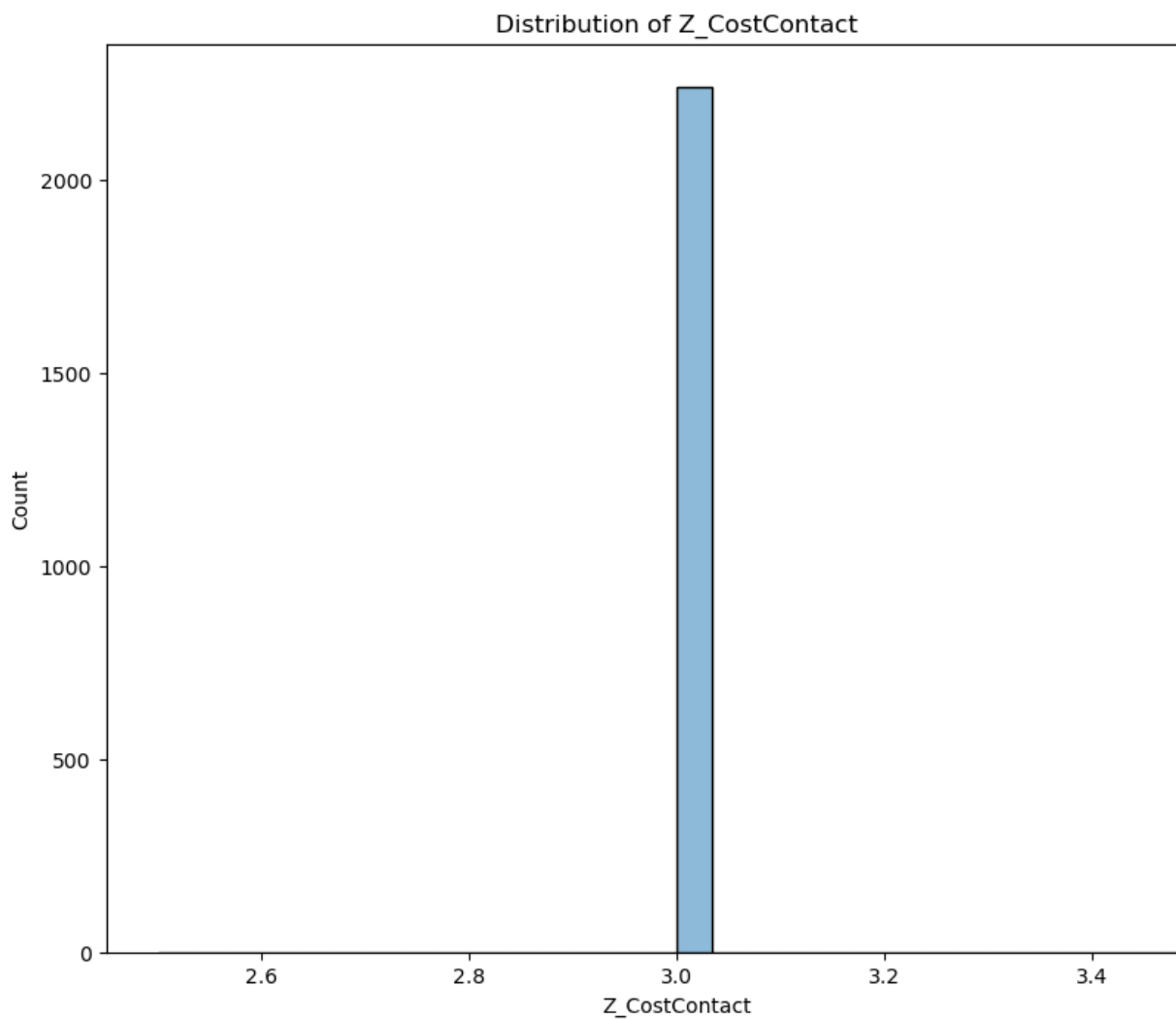
```
In [22]: df['Income'], fitted_lambda = boxcox(df['Income'])
print("Lambda:", fitted_lambda)
```

```
Lambda: 0.46443202095696146
```

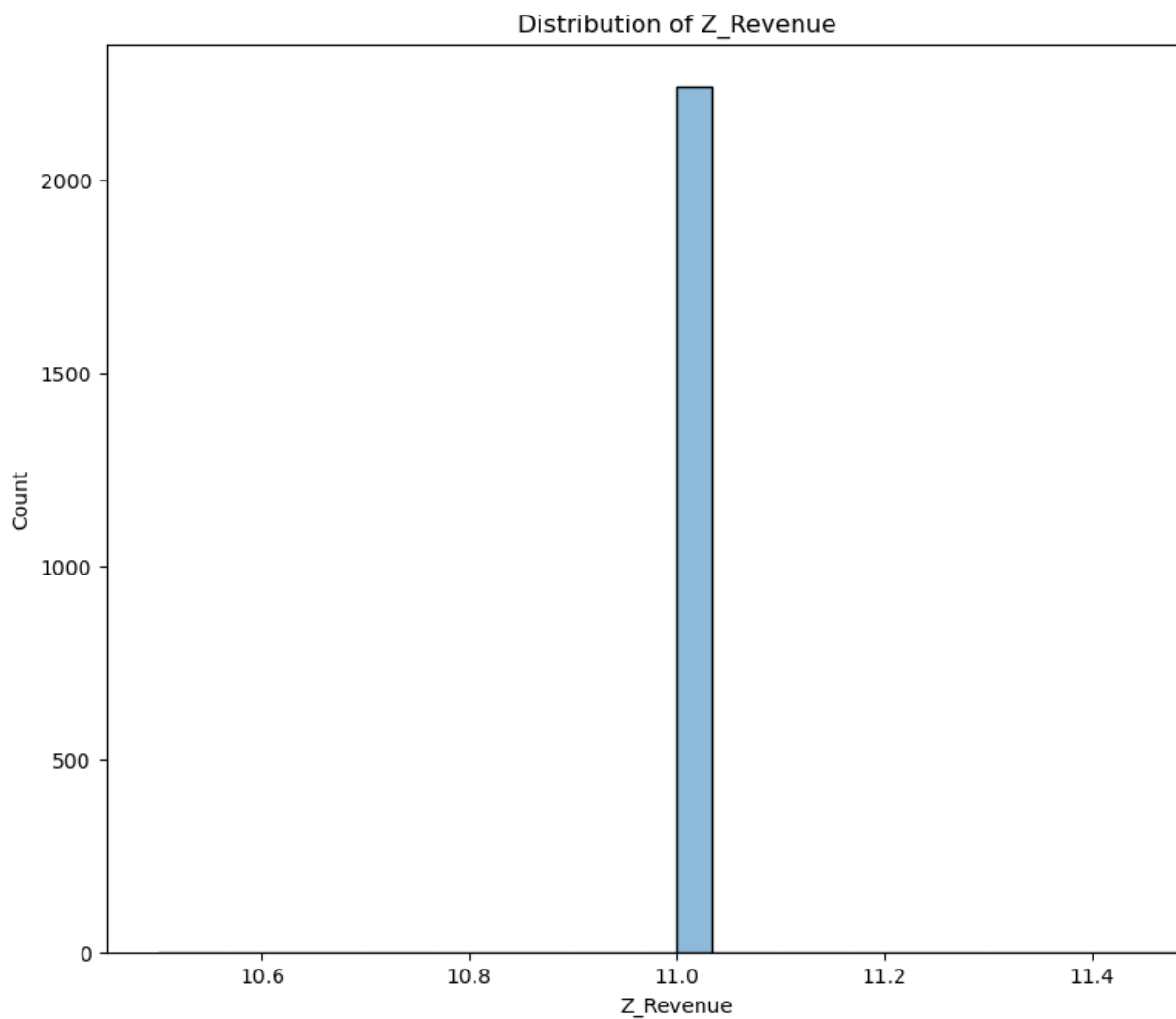
```
In [23]: df['Income'].skew() #skewness is removed using box cox transformation beacause L
```

```
Out[23]: 0.233020844663721
```

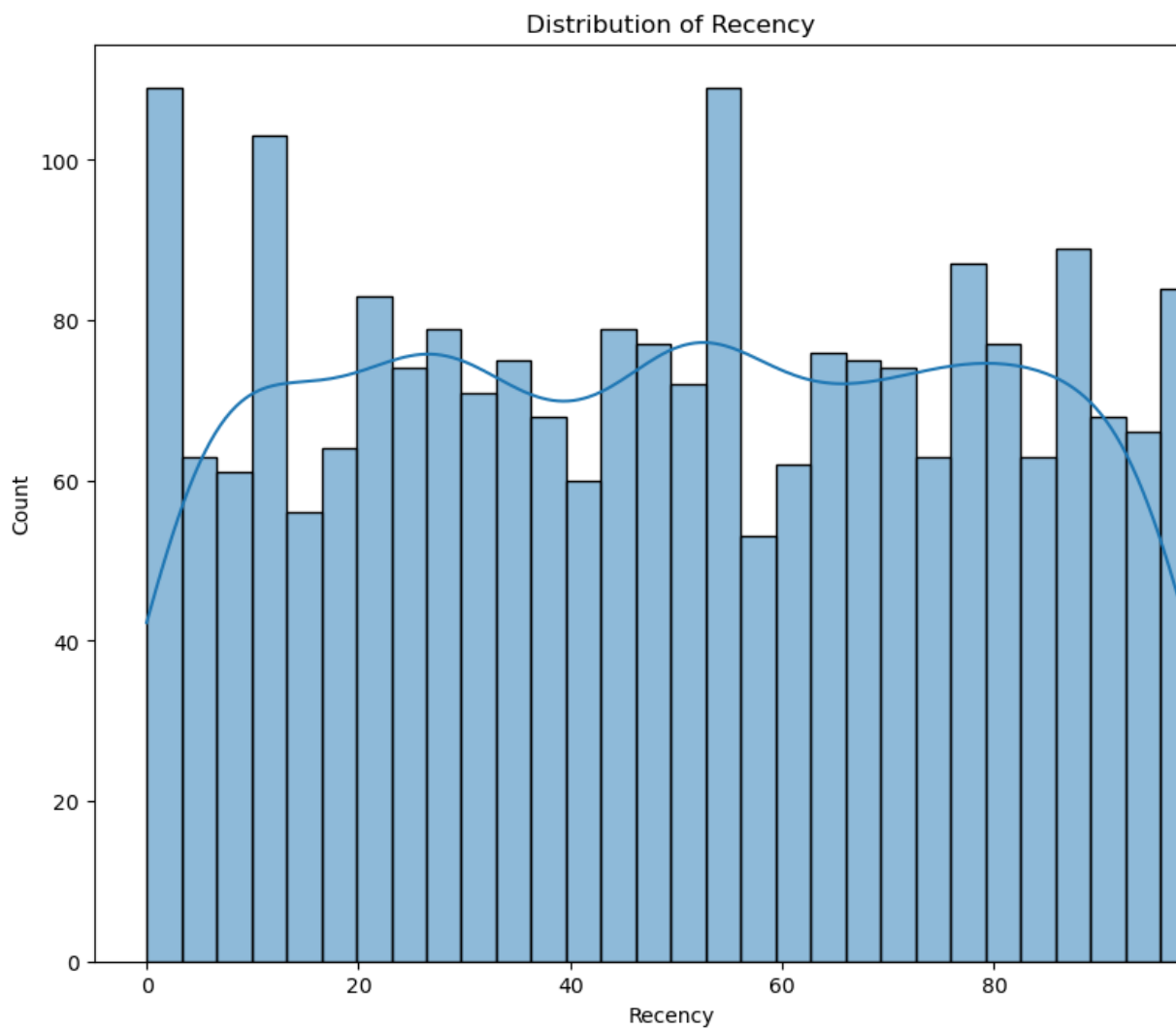
```
In [24]: plt.figure(figsize=(10,8))
sns.histplot(df["Z_CostContact"] , bins = 30, kde = True)
plt.title("Distribution of Z_CostContact") ##It is just a constant value
plt.xlabel("Z_CostContact")
plt.ylabel("Count")
plt.show()
```



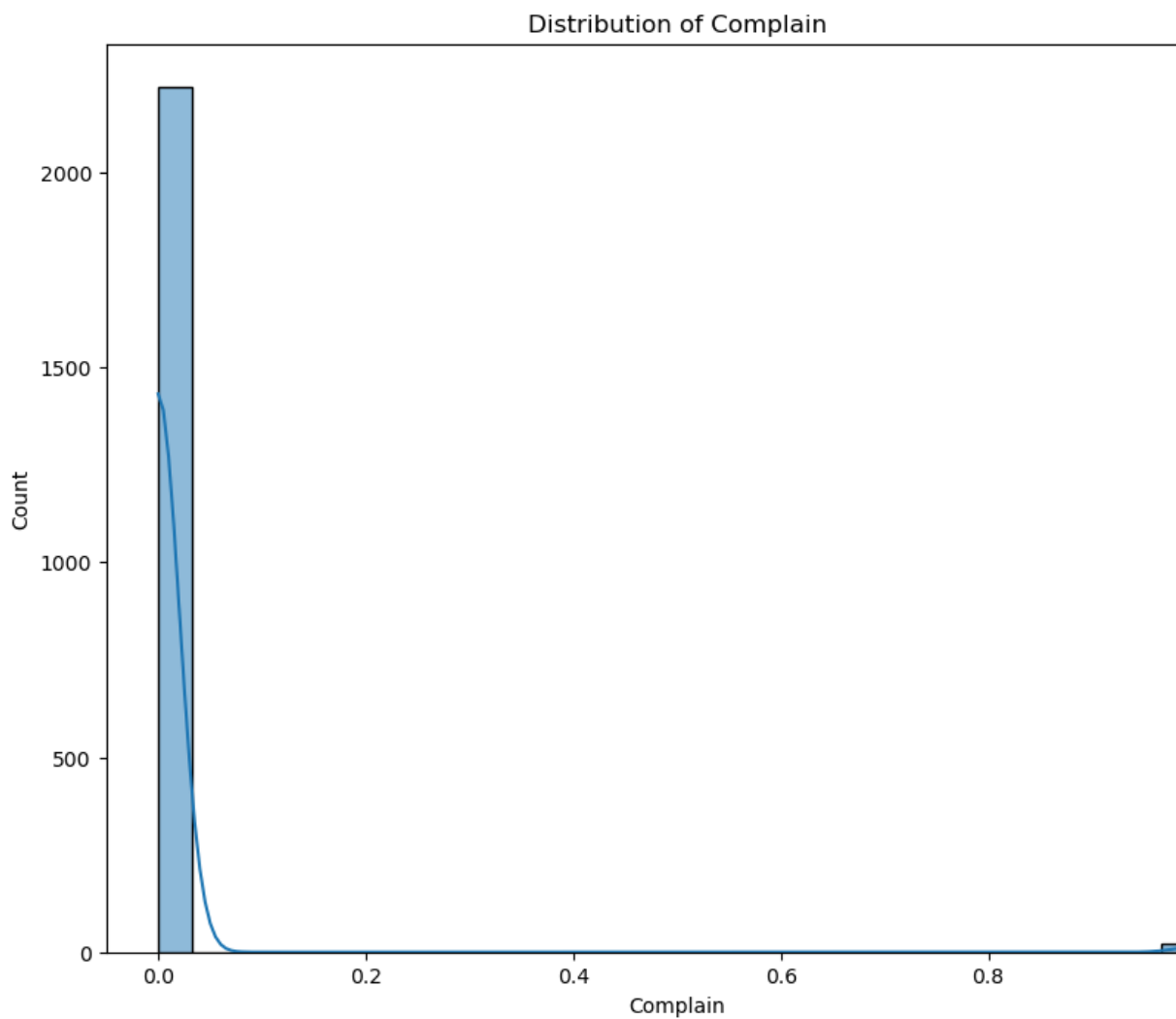
```
In [25]: plt.figure(figsize=(10,8))
sns.histplot(df["Z_Revenue"] , bins = 30, kde = True)
plt.title("Distribution of Z_Revenue") #It is just a constant
plt.xlabel("Z_Revenue")
plt.ylabel("Count")
plt.show()
```



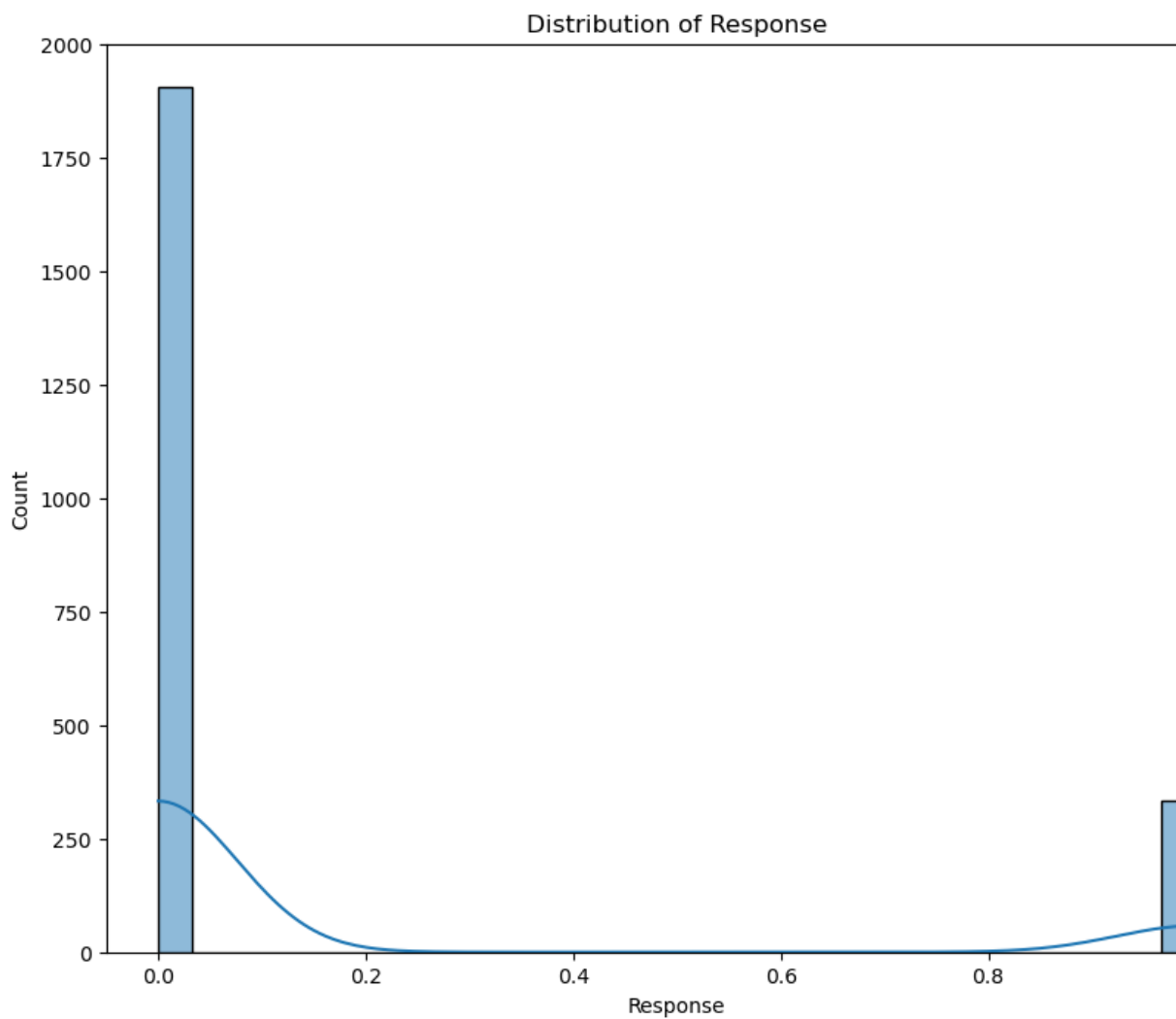
```
In [26]: plt.figure(figsize=(10,8))
sns.histplot(df["Recency"] , bins = 30, kde = True)
plt.title("Distribution of Recency")
plt.xlabel("Recency")
plt.ylabel("Count")
plt.show()
```



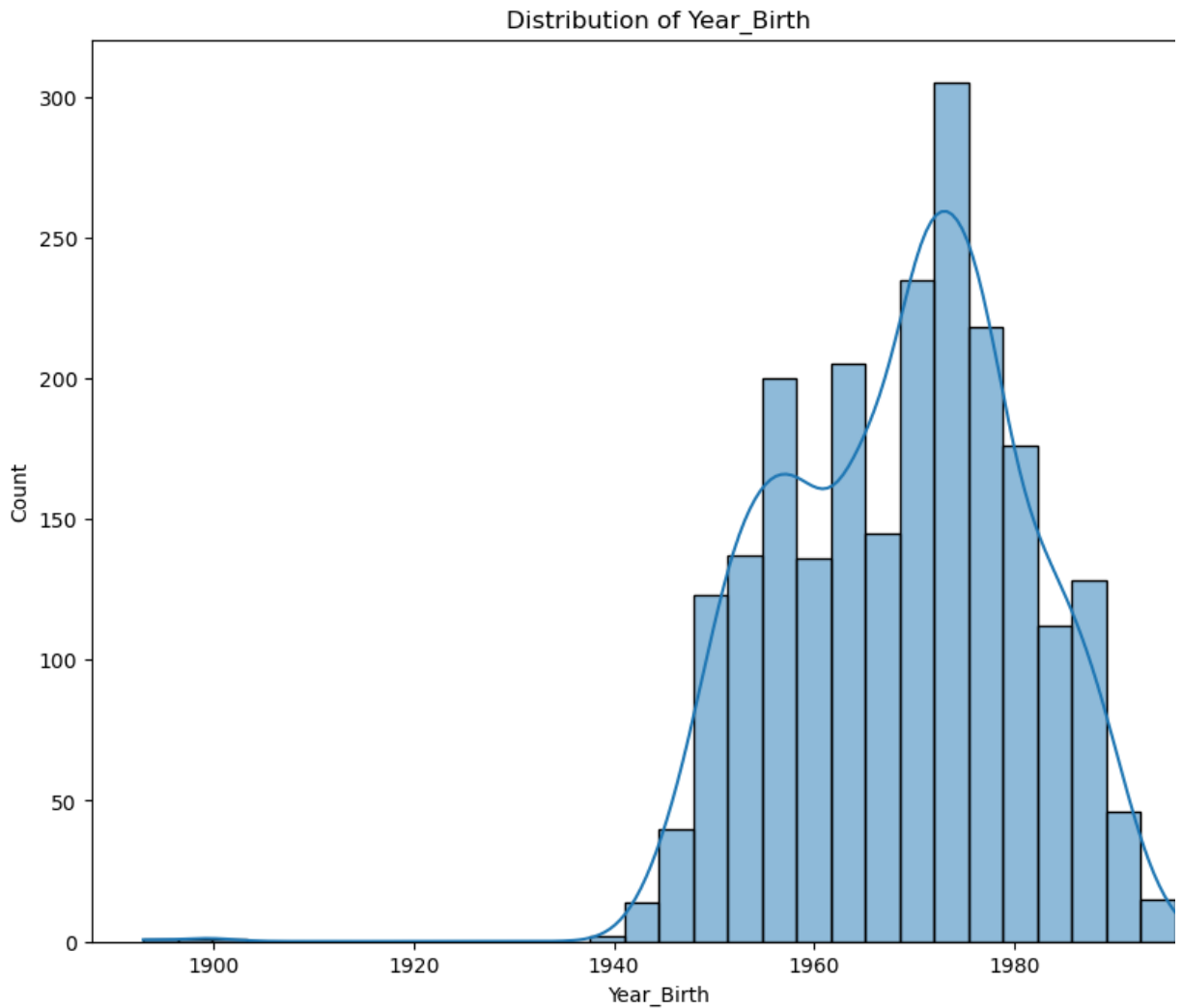
```
In [27]: plt.figure(figsize=(10,8))
sns.histplot(df["Complain"] , bins = 30, kde = True)
plt.title("Distribution of Complain")
plt.xlabel("Complain")
plt.ylabel("Count")
plt.show()
```



```
In [28]: plt.figure(figsize=(10,8))
sns.histplot(df["Response"] , bins = 30, kde = True)
plt.title("Distribution of Response")
plt.xlabel("Response")
plt.ylabel("Count")
plt.show()
```



```
In [29]: plt.figure(figsize=(10,8))
sns.histplot(df["Year_Birth"] , bins = 30, kde = True)
plt.title("Distribution of Year_Birth")
plt.xlabel("Year_Birth")
plt.ylabel("Count")
plt.show()
```



Data Preprocessing and Data Engineering

```
In [31]: df['Response'] = df['Response'].astype('category')
```

```
In [32]: df['Marital_Status'] = df['Marital_Status'].astype('category')
```

```
In [33]: from scipy.stats import chi2_contingency
```

```
In [34]: connection = pd.crosstab(df['Marital_Status'], df['Response']) #we cannot drop n
chi2 , p , dof , expected =chi2_contingency(connection)
print("p value:" , p )
```

p value: 2.1067470540906996e-09

```
In [35]: df["Response"].corr(df["ID"])
```

Out[35]: -0.021967817648744652

```
In [36]: df = df.drop("ID", axis = 1)
```

```
In [37]: from datetime import datetime

current_year= datetime.now().year
df["age"] = current_year - df["Year_Birth"]

df = df.drop(["Year_Birth"], axis = 1)
```

```
In [38]: df['KidsTotal'] = df["Kidhome"] + df["Teenhome"]
df= df.drop(["Kidhome","Teenhome"], axis=1) #we have combined kidhome and teenhome
df
```

```
Out[38]:
```

	Education	Marital_Status	Income	Dt_Customer	Recency	MntWines	MntFruits	Mr
0	Graduation	Single	349.282367	2012-09-04	58	635	88	
1	Graduation	Single	314.158086	2014-03-08	38	11	1	
2	Graduation	Together	385.007767	2013-08-21	26	426	49	
3	Graduation	Together	242.461824	2014-02-10	26	11	4	
4	PhD	Married	349.717207	2014-01-19	94	173	43	
...	...	...	...	...	...	...	...	...
2235	Graduation	Married	357.823432	2013-06-13	46	709	43	
2236	PhD	Together	365.354023	2014-06-10	56	406	0	
2237	Graduation	Divorced	346.016691	2014-01-25	91	908	48	
2238	Master	Together	379.008502	2014-01-24	8	428	30	
2239	PhD	Married	334.113388	2012-10-15	40	84	3	

2240 rows × 28 columns

```
In [39]: df["Dt_Customer"] = pd.to_datetime(df["Dt_Customer"],errors= "coerce",dayfirst=True)
```

```
In [40]: cat_cols = ["Education","Marital_Status","Complain"]
df[cat_cols] = df[cat_cols].astype("category")
```

```
In [41]: df["TotalAccCmp"] = df["AcceptedCmp5"] + df["AcceptedCmp1"] + df["AcceptedCmp2"]
```

```
In [42]: df= df.drop(["AcceptedCmp5","AcceptedCmp1","AcceptedCmp2","AcceptedCmp3","AcceptedCmp4"])
```

```
In [43]: df["TotalMntspent"] = df["MntWines"] + df["MntFruits"] + df["MntMeatProducts"] + df["MntSeafoodProducts"]
df = df.drop(["MntWines","MntFruits","MntMeatProducts","MntFishProducts","MntSweetProducts","MntOtherProducts"])
```

```
In [44]: df["TotalNumPurchases"] = df["NumDealsPurchases"] + df["NumWebPurchases"] + df["NumCatalogPurchases"]
```

```
In [45]: df= df.drop(["NumDealsPurchases","NumWebPurchases","NumCatalogPurchases","NumStorePurchases"])
```

```
In [46]: df[["Z_Revenue","Z_CostContact","Response"]].corr()
```

```
Out[46]:
```

	Z_Revenue	Z_CostContact	Response
Z_Revenue	NaN	NaN	NaN
Z_CostContact	NaN	NaN	NaN
Response	NaN	NaN	1.0

```
In [47]: df= df.drop(columns=["Z_Revenue","Z_CostContact"])
```

```
In [48]: df[["income","Response"]].corr()
```

Out[48]:

	income	Response
income	1.000000	0.126492
Response	0.126492	1.000000

In [49]: `df["income"].isnull().sum()`

Out[49]: 0

In [50]: `df["Income"].equals(df["income"])`

Out[50]: False

In [51]: `df[["income", "Income"]].corr()`

Out[51]:

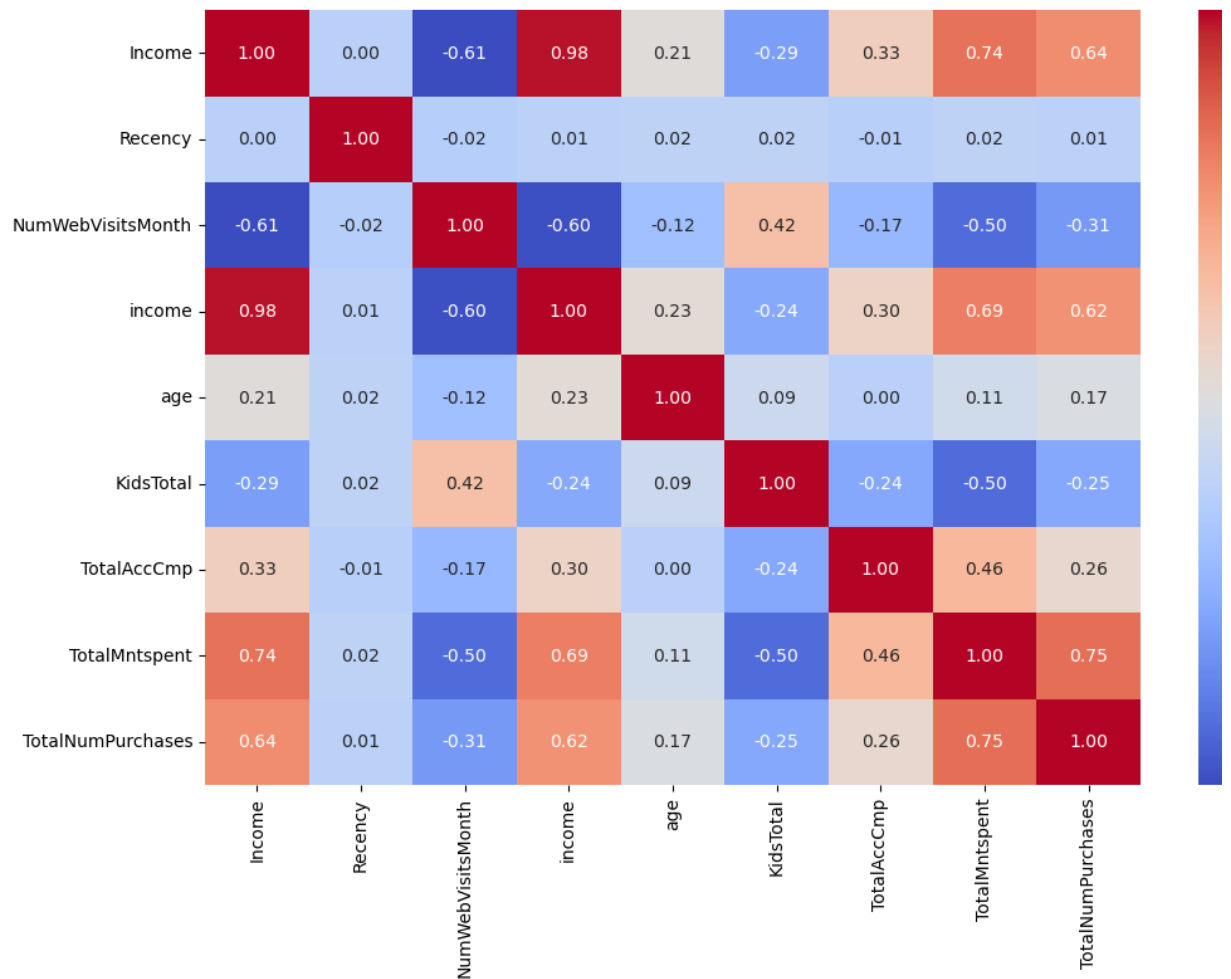
	income	Income
income	1.000000	0.975793
Income	0.975793	1.000000

In [52]: `df[["income", "Response"]].corr()`
`df[["Income", "Response"]].corr()`

Out[52]:

	Income	Response
Income	1.000000	0.140086
Response	0.140086	1.000000

In [53]: `corr_matrix = df.select_dtypes(include=["number"]).corr()`
`plt.figure(figsize=(12,8))`
`sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm")`
`plt.show()`



```
In [54]: df= df.drop(columns=["income"], axis = 1)
```

```
In [55]: df
```

```
Out[55]:
```

	Education	Marital_Status	Income	Dt_Customer	Recency	NumWebVisitsMonth	Cor
0	Graduation	Single	349.282367	2012-04-09	58	7	
1	Graduation	Single	314.158086	2014-08-03	38	5	
2	Graduation	Together	385.007767	NaT	26	4	
3	Graduation	Together	242.461824	2014-10-02	26	6	
4	PhD	Married	349.717207	NaT	94	5	
...	...	...	...	...	...	...	
2235	Graduation	Married	357.823432	NaT	46	5	
2236	PhD	Together	365.354023	2014-10-06	56	7	
2237	Graduation	Divorced	346.016691	NaT	91	6	
2238	Master	Together	379.008502	NaT	8	3	
2239	PhD	Married	334.113388	NaT	40	7	

2240 rows × 13 columns

```
In [ ]:
```

```
In [56]: df["Dt_Customer"].isnull().sum()
```

Out[56]: 1324

```
In [57]: df.isna().sum()
```

```
Out[57]: Education          0
Marital_Status            0
Income                   0
Dt_Customer              1324
Recency                  0
NumWebVisitsMonth        0
Complain                 0
Response                 0
age                     0
KidsTotal                0
TotalAccCmp             0
TotalMntspent           0
TotalNumPurchases       0
dtype: int64
```

```
In [58]: df["dt_customer_missing"] = df["Dt_Customer"].isna().astype(int)
```

```
In [59]: df["dt_customer_missing"].value_counts()
```

```
Out[59]: dt_customer_missing
1      1324
0       916
Name: count, dtype: int64
```

```
In [60]: op = df["Dt_Customer"].median().date()
op
```

```
Out[60]: datetime.date(2013, 7, 1)
```

```
In [61]: df['Dt_Customer'] = df['Dt_Customer'].fillna(op)
```

```
In [62]: df['Dt_Customer'] = pd.to_datetime(df['Dt_Customer'], errors='coerce')
df['year'] = df['Dt_Customer'].dt.year
df['month'] = df['Dt_Customer'].dt.month
df['day'] = df['Dt_Customer'].dt.day
```

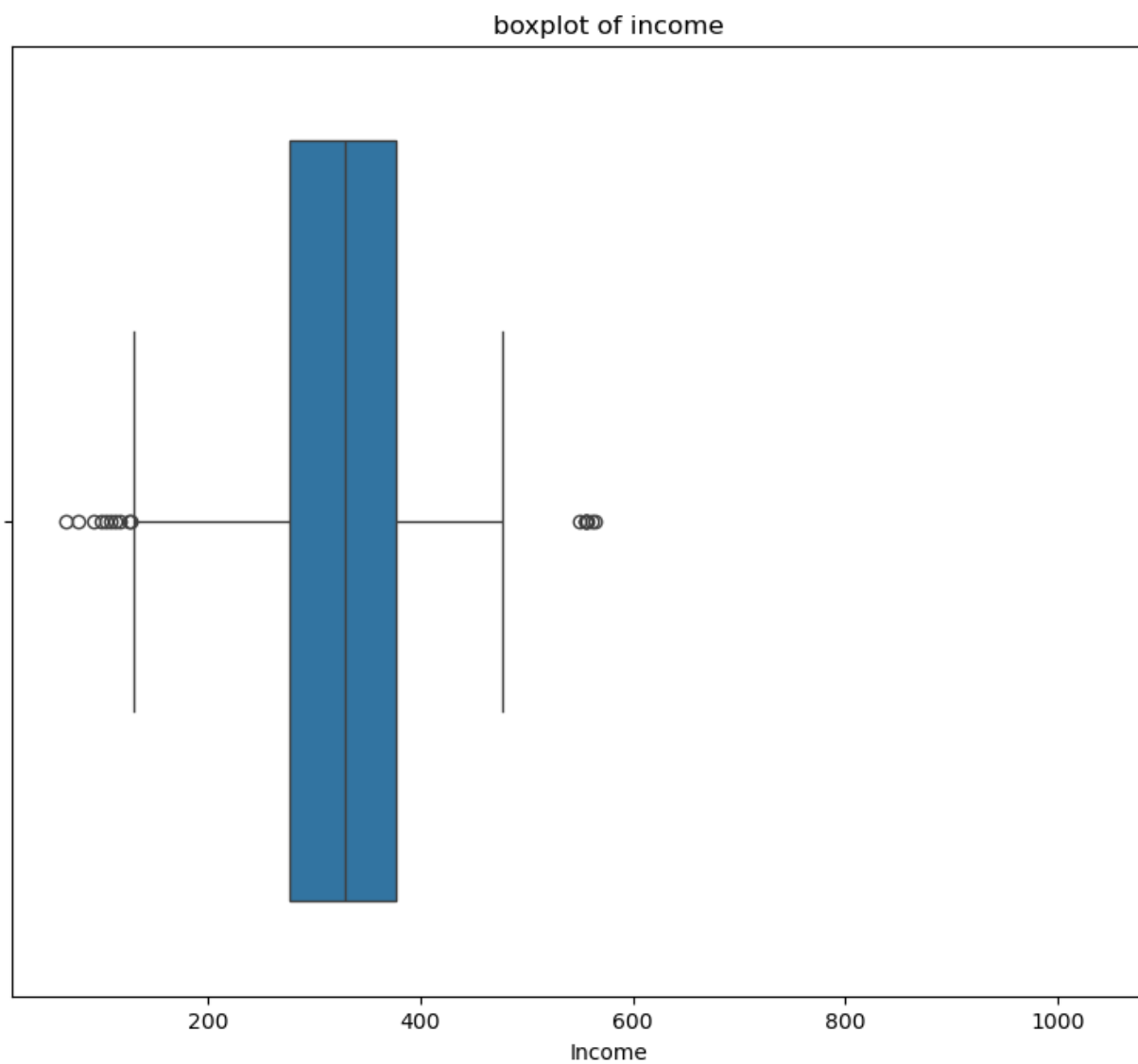
```
In [63]: df = df.drop(columns=['Dt_Customer'])
df
```

Out[63]:

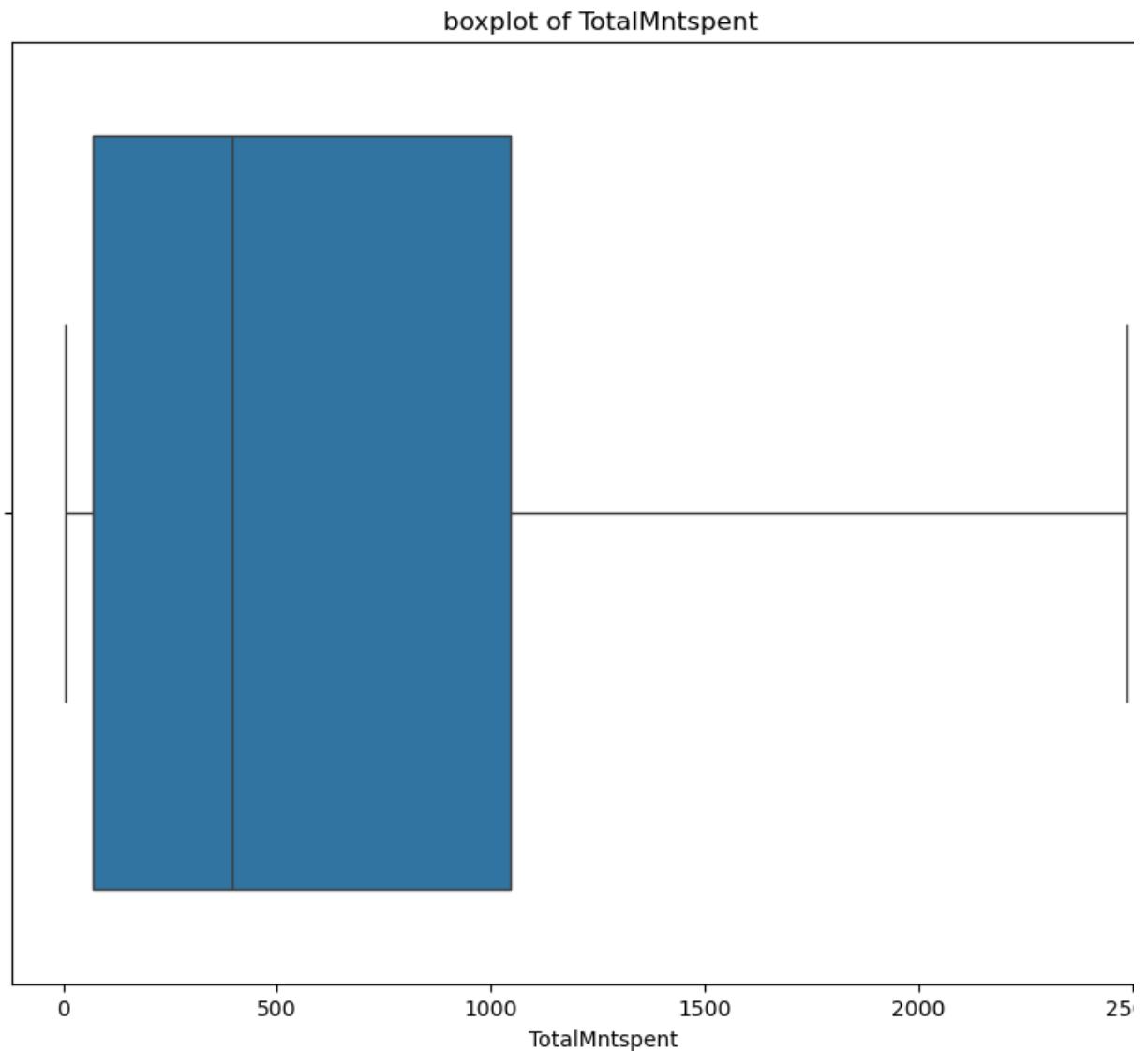
	Education	Marital_Status	Income	Recency	NumWebVisitsMonth	Complain	Respor
0	Graduation	Single	349.282367	58	7	0	
1	Graduation	Single	314.158086	38	5	0	
2	Graduation	Together	385.007767	26	4	0	
3	Graduation	Together	242.461824	26	6	0	
4	PhD	Married	349.717207	94	5	0	
...	...	...	...	...	...	...	...
2235	Graduation	Married	357.823432	46	5	0	
2236	PhD	Together	365.354023	56	7	0	
2237	Graduation	Divorced	346.016691	91	6	0	
2238	Master	Together	379.008502	8	3	0	
2239	PhD	Married	334.113388	40	7	0	

2240 rows × 16 columns

```
In [64]: plt.figure(figsize=(10,8))
sns.boxplot(x=df['Income'])
plt.title('boxplot of income')
plt.show()
```



```
In [65]: plt.figure(figsize=(10,8))
sns.boxplot(x=df['TotalMntspent'])
plt.title('boxplot of TotalMntspent ')
plt.show()
```



```
In [66]: Q1 = df['Income'].quantile(0.25)
Q3 = df['Income'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print('lower_bound:', lower_bound)
print('upper_bound:', upper_bound)
```

```
lower_bound: 128.83334294009325
upper_bound: 525.1917277666698
```

```
In [67]: df['Income'] = np.where(df['Income'] > upper_bound, upper_bound,
                                np.where(df['Income'] < lower_bound, lower_bound, df['Income']))
```

```
In [ ]:
```

```
In [68]: y = df['Response']
X = df.drop(columns=['Response'])

# Identify numeric and categorical columns
num_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = X.select_dtypes(include=['object', 'category']).columns.tolist()
```

```
In [69]: num_transformer = Pipeline(steps=[
        ('scaler', StandardScaler())
    ])

    cat_transformer = Pipeline(steps=[
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
    ])
```

```
In [70]: preprocessor = ColumnTransformer(
        transformers=[
            ('num', num_transformer, num_cols),
            ('cat', cat_transformer, cat_cols)
        ],
        remainder='drop'
    )
```

```
In [71]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

```
In [72]: print("Train shape:", X_train.shape)
        print("Test shape:", X_test.shape)
```

Train shape: (1792, 15)
Test shape: (448, 15)

```
In [73]: # Define models
        models = {
            "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balar",
            "Decision Tree": DecisionTreeClassifier(random_state=42, class_weight="balar",
            "Random Forest": RandomForestClassifier(random_state=42, class_weight="balar",
            "KNN": KNeighborsClassifier(n_neighbors=5),
            "Bagging (DT)": BaggingClassifier(estimator=DecisionTreeClassifier(class_w
                n_estimators=100, random_state=42),
            "AdaBoost": AdaBoostClassifier(n_estimators=100, random_state=42),
            "Gradient Boosting": GradientBoostingClassifier(n_estimators=100, random_sta
        }

        results = {}

        plt.figure(figsize=(10, 8))

        for name, model in models.items():
            pipe = Pipeline(steps=[
                ("preprocessor", preprocessor),
                ("classifier", model)
            ])

            # Cross-validation
            cv_scores = cross_val_score(pipe, X, y, cv=5, scoring="accuracy")
            cv_mean = cv_scores.mean()

            # Train on training data
            pipe.fit(X_train, y_train)
            y_pred = pipe.predict(X_test)
            y_proba = pipe.predict_proba(X_test)[:, 1] if hasattr(pipe, "predict_proba")

            acc = accuracy_score(y_test, y_pred)
            auc = roc_auc_score(y_test, y_proba) if y_proba is not None else None

            results[name] = {
```

```

        "Test Accuracy": acc,
        "CV Mean Accuracy": cv_mean,
        "ROC-AUC": auc
    }

    print(f"◆ {name}")
    print("Accuracy:", acc)
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))
    print("-"*60)

    # ROC curve
    if y_proba is not None:
        RocCurveDisplay.from_predictions(y_test, y_proba, name=name, ax=plt.gca())

plt.plot([0, 1], [0, 1], "k--")
plt.title("ROC Curves")
plt.show()

# Summary DataFrame
results_df = pd.DataFrame(results).T.sort_values(by="ROC-AUC", ascending=False)
print("\n Model Comparison (CV + ROC-AUC):")
print(results_df)

```

◆ Logistic Regression

Accuracy: 0.7924107142857143

Confusion Matrix:

```
[[304  77]
```

```
[ 16  51]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.80	0.87	381
1	0.40	0.76	0.52	67
accuracy			0.79	448
macro avg	0.67	0.78	0.70	448
weighted avg	0.87	0.79	0.82	448

◆ Decision Tree

Accuracy: 0.8459821428571429

Confusion Matrix:

```
[[349  32]
```

```
[ 37  30]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.92	0.91	381
1	0.48	0.45	0.47	67
accuracy			0.85	448
macro avg	0.69	0.68	0.69	448
weighted avg	0.84	0.85	0.84	448

◆ Random Forest

Accuracy: 0.8794642857142857

Confusion Matrix:

```
[[376   5]
```

```
[ 49  18]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.99	0.93	381
1	0.78	0.27	0.40	67
accuracy			0.88	448
macro avg	0.83	0.63	0.67	448
weighted avg	0.87	0.88	0.85	448

◆ KNN

Accuracy: 0.8794642857142857

Confusion Matrix:

```
[[368  13]
```

```
[ 41  26]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.97	0.93	381
1	0.67	0.39	0.49	67
accuracy			0.88	448

macro avg	0.78	0.68	0.71	448
weighted avg	0.86	0.88	0.87	448

◆ Bagging (DT)

Accuracy: 0.8772321428571429

Confusion Matrix:

[[368 13]

[42 25]]

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.97	0.93	381
1	0.66	0.37	0.48	67
accuracy			0.88	448
macro avg	0.78	0.67	0.70	448
weighted avg	0.86	0.88	0.86	448

◆ AdaBoost

Accuracy: 0.8816964285714286

Confusion Matrix:

[[371 10]

[43 24]]

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.97	0.93	381
1	0.71	0.36	0.48	67
accuracy			0.88	448
macro avg	0.80	0.67	0.70	448
weighted avg	0.87	0.88	0.86	448

◆ Gradient Boosting

Accuracy: 0.875

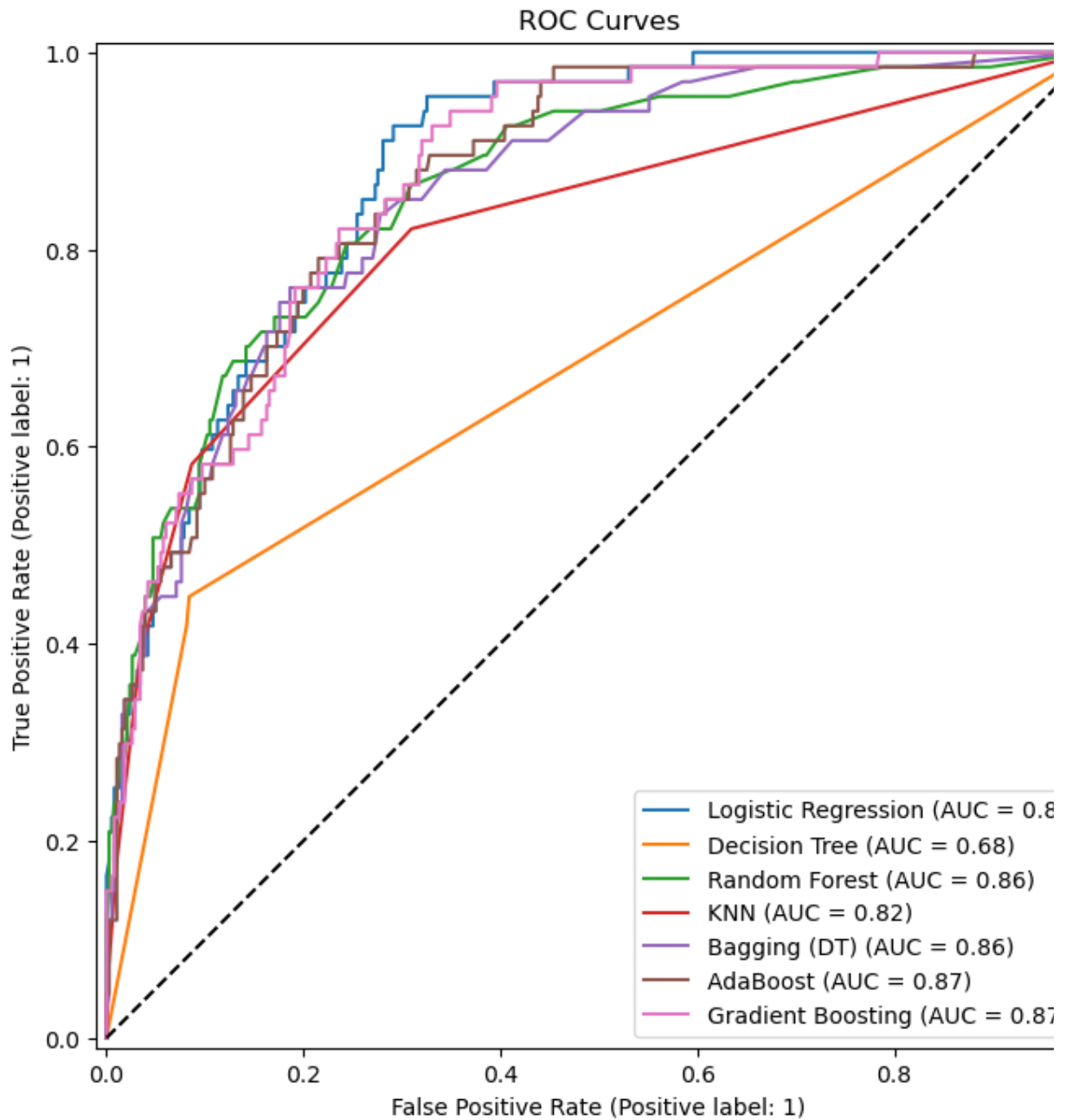
Confusion Matrix:

[[372 9]

[47 20]]

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.98	0.93	381
1	0.69	0.30	0.42	67
accuracy			0.88	448
macro avg	0.79	0.64	0.67	448
weighted avg	0.86	0.88	0.85	448



Model Comparison (CV + ROC-AUC):

	Test Accuracy	CV Mean Accuracy	ROC-AUC
Logistic Regression	0.792411	0.791518	0.881831
Gradient Boosting	0.875000	0.883482	0.871685
AdaBoost	0.881696	0.877679	0.866514
Random Forest	0.879464	0.876339	0.861284
Bagging (DT)	0.877232	0.876339	0.856622
KNN	0.879464	0.873661	0.818232
Decision Tree	0.845982	0.825446	0.681220

Conclusion

- If your business wants to catch as many responders as possible (maximize recall) → Logistic Regression.
- If your business wants higher targeting accuracy (maximize precision, reduce cost) → Gradient Boosting.
- If you want a balanced model for production → AdaBoost is the sweet spot.

In []:

In []:

In []:

In []: